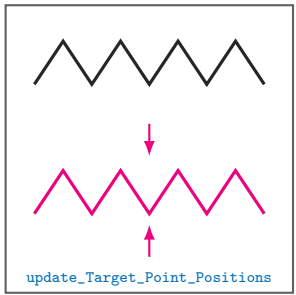
MOVING BOUNDARIES, AND MAKING YOUR OWN EXAMPLE

The `update_*` functions that move target points, change springs and bend beams in time; the moving band and the sawtooth pump, run in Python; the two styles of prescribing motion; a geometry of your own from a sketch to a running folder



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part II • Chapter 4c

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

WHAT YOU WILL BE ABLE TO DO AFTER THIS CHAPTER

- Name the four ways a boundary moves in IB2d (target points, springs, beams, muscles), the `input2d` flag and the function file behind each, and what the driver passes to that function.
- Write an `update_Target_Point_Positions` function: the moving band (every point dragged along a path), the sawtooth pump (one row of a channel driven in time) and the pulsing heart (every point blended between two shapes), all three in Python.
- Choose between the two ways of prescribing a motion (increment the targets every step, or set them from the original positions) and know why the second one does not drift.
- Change spring resting lengths and beam curvatures in time with the same mechanism (the impedance pump and the swimmer of chapter 4b).
- Build an example of your own from a sketch: domain, grid, points at $\Delta x/2$, indices, the files, the flags, a short first run, the checks.

> Three examples were ported to Python and run in October 2026 (`examples/IB2d/Moving_Rubberband`, `Sawtooth`, `Pulsing_Heart` in the repository); the ports reproduce the MATLAB geometry files to round-off ($2 \cdot 10^{-15}$). The MATLAB sawtooth is a 1024×256 run of hours and did not survive on any laptop grid tried (slide 14): it is the chapter's cautionary case; the pulsing heart is its working pump.

WHERE THIS CHAPTER COMES FROM

- "Changing target positions" (Lecture 3 of the 2022 UA tutorials, 20 slides; its IB2d half, slides 3–13): the four ways to move a boundary, the sawtooth example line by line (`give_Me_Sawtooth_Geometry`, what it prints, `update_Target_Point_Positions`, `N_top`), and the IBAMR half that is chapter 9a.
- Lectures 11 and 12 (2021–23): the same sawtooth block, the homework, and (Lecture 12) the stability material that is chapter 4d; the 2024 practical's slide 29 repeats the four-way list.
- "IB2d and IBAMR associated files" (Lecture 2, 2022), slides 20–24 and 32–38: creating geometries ($\Delta s = \Delta x/2$, the origin, fitting the box), the rubber-band generator in both flavours, the MATLAB checkers, the project ideas.
- The course's "4-ib2d-MakeYourOwnExample" folder (two beams driven by target points) and its homework ("Your own rubber band", Lectures 3 and 9; option 2 of Lecture 11's homework 4).
- The IB2d examples `Example_Moving_Rubberband`, `Example_Sawtooth`, `Example_Pulsing_Heart` (all three ported here), `Tracers_In_Impedance_Pump` (Python) and the swimmer (chapter 4b).

What is new here: the three Python ports and their runs, the two prescription styles, the sawtooth's failure on the coarser grids tried on a desktop PC, the checklist for a new example, and three things to fix in the course's own template.

FOUR WAYS TO MOVE A BOUNDARY, ONE MECHANISM

Move it by	`input2d`	the function the driver calls every step	the table it edits
target points that travel	`target_pts = 1`, `update_target = 1`	`update_Target_Point_Positions(dt, current_time, targets)`	index, x, y, k_t
springs whose k or L change	`springs = 1`, `update_springs = 1`	`update_Springs(dt, current_time, xLag, yLag, springs)`	i, j, k, L, α
beams whose rest curvature changes	`beams = 1`, `update_beams = 1` (or the `nonInv` pair)	`update_Beams(dt, current_time, beams)`, `update_nonInv_Beams(...)`	i, j, k, k_b, C (or C_x, C_y)
damped springs	`damped_springs = 1`, `update_damp_springs = 1`	`update_Damped_Springs(dt, current_time, d_springs)`	i, j, k, L, b
muscles	`FV_LT_muscle = 1`	an activation function you write (chapter 4e)	

HOW THE DRIVER USES THE UPDATE FUNCTIONS

- The function lives in the example folder, `.m` for MATLAB, `.py` for Python (the Python driver does `from update_Springs import update_Springs` inside the step loop). The driver calls it once per step, after the half-step move of the points and before the forces are computed, and replaces the table with what the function returns.
- The structure's own positions are never set by these functions: a target moves, and the point follows it through the target force $k_t(\mathbf{X}^{\text{target}} - \mathbf{X})$ (chapter 4a); a resting length changes, and the spring pulls. That is why the motion is "approximately prescribed" and why the stiffness matters (slide 14).
- Lecture 3's list: "target points to provide a preferred motion; springs whose stiffness or resting lengths change in time; beams whose resting curvature changes in time; muscle models / muscle springs"; Lectures 11 and 12 add "This is typically done by making an `update_XXX.m` or `.py` file."

THE TARGET TABLE

column	MATLAB (1-based) / Python (0-based)
1 / 0	the point's index in the `.vertex` order
2 / 1	the target's x
3 / 2	the target's y
4 / 3	the stiffness k_t from the `.target` file

- Built by the driver from the `.target` file and the initial `.vertex` positions, in the order of the `.target` file; the table's row r is the target of point `targets(r, 1)`.
- The examples write a `.target` line for every point in order, so row r is point r; a file that lists a subset of points, or in another order, breaks that habit, and the index column is there for it.
- Change columns 2–3 (MATLAB) or 1–2 (Python) and return the table: the targets have moved. Change column 4/3 and the tether stiffens or loosens (a way to release a structure at a given time).
- `current_time` is the time at the start of the step; `dt` the step. A motion written as a function of `current_time` is exact; one written as an increment per step accumulates (slide 9).
- Lecture 3: "you need to loop through all vertices and determine which to move (1–3276) and which to not move (3277–6552)", which is why the generator prints the counts (slide 10).

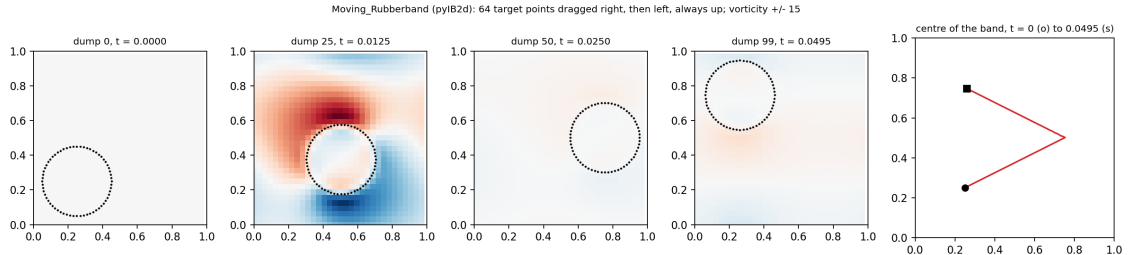
EXAMPLE 1: THE MOVING BAND (EXAMPLE_MOVING_RUBBERBAND, IN PYTHON)

```
def update_Target_Point_Positions(dt,
     current_time, targets):
    period = 0.05              # time of one right-
     or-left leg
    L = 0.5                    # distance travelled
      in each direction
    s = L / (period / 2)       # speed
    t = current_time % period
    targets[:, 2] += (s / 2) * dt          # up,
     always
    if t < period / 2:
        targets[:, 1] += s * dt            # right
    else:
        targets[:, 1] -= s * dt            # left
    return targets
```

The MATLAB original loops over the rows and does the same (`targets(IDs(i),3) = yPts(IDs(i)) + (s/2)*dt` and the x branch); the Python port uses the table's columns at once.

- A circle of 64 target points ($k_t = 10^7$) centred at (0.25, 0.25), in a viscous fluid ($\mu = 10$, $\rho = 1$), 32^2, $\Delta t = 10^{-4}$, $T = 0.05$, a dump every 5 steps.
- Every step the targets move right (first half of the period) or left (second half) at speed 20 and up at speed 10: a zig-zag of two legs of 0.5.
- Files: `Rubberband.py` (the generator: `.vertex` and `.target`, 0-based), `update_Target_Point_Positions.py`, `input2d` with `target_pts = 1` and `update_target = 1`, `main2d.py`.
- 2.9 s for 500 steps; 100 dumps.

THE MOVING BAND: WHAT HAPPENS



- The band's centre goes from (0.25, 0.25) to (0.75, 0.50) at $t = 0.025$ and back to (0.26, 0.75) at the end: the two legs of the zig-zag (right panel). The points keep up with their targets because k_t is large and the fluid is viscous; the vorticity (± 15 in the frames shown) is the wake of a cylinder dragged through a viscous fluid; the reversal of the target velocity at $t = 0.025$ produces a spike of 840 one dump later that decays within a few dumps: an instantaneous change of a prescribed velocity is a jolt, and a smooth reversal (slide 19) would avoid it.
- Nothing in the `.vertex` file knows about the motion: the same files with `update_target = 0` give a band that sits still. The motion is in the function, and the fluid responds.

TWO WAYS TO PRESCRIBE A MOTION

Increment every step (the moving band):

```
targets[:, 1] += s * dt
```

- Short to write; the motion is whatever the increments add up to.
- The position after n steps is $n\,s\,\Delta t$ in floating point, and the leg changes direction when `t < period/2` flips, which depends on how `current_time` is accumulated; change Δt and the turning point moves by up to one step. Over many periods the table drifts from where the formula says.

Set from the original positions (the sawtooth):

```
shift = (a / 2) * sin(2 * pi * freq *
     current_time)
for i in range(N_top):
    targets[i, 2] = y0[i] - shift
```

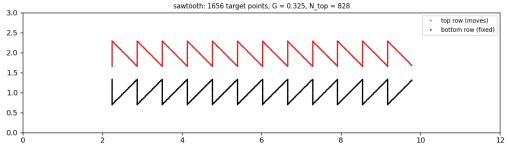
- The targets are a function of time and of the original geometry (`y0`, read once from the `.vertex` file): no drift, the same motion at every Δt, and a restart from any time is exact.
- Lecture 3: “this code is written such that the new positions are calculated using the original positions. You could also modify based on the position from the previous time step.” Prefer the first.

Both styles are fine for a demonstration; for a study (a resolution or time-step sweep, chapter 4d) use the second, so that the prescribed motion is identical across the runs you compare.

EXAMPLE 2: THE SAWTOOTH PUMP: THE GEOMETRY

```
Lx = 12;  Ly = 3.0;              % the box
Nx = 1024;  Ny = 256;            % chosen so that dx = dy
ds = Lx/(2.1*Nx);                % a little under dx/2
d = 0.63;  w = 0.63;             % tooth height and width
G = 0.325;                       % vertical gap between the rows
[xLag, yLag] = give_Me_Lagrangian_Geometry(ds,d,w,G);
xLag = xLag + ( Lx/2 - xLag(end/4) );  % centre in x
yLag = yLag + ( Ly/2 );                % centre in y
fprintf('G = %d (vertical gap btwn top and bottom)\n',G);
fprintf('N_Top = %d (# of points on top of sawtooth)\n', length(
      xLag)/2);
k_Target = 1e11;
print_Lagrangian_Target_Pts(xLag,k_Target,struct_name);
```

`give_Me_Sawtooth_Geometry.m`, abridged. One tooth is a vertical rise of d followed by a diagonal down to $(w, 0)$; 12 teeth make the top row at $+G/2$, their mirror image the bottom row at $-G/2$.



The generator's own figure at 256×64: the top row (red) is the one that moves.

THE SAWTOOTH'S NUMBERS

- 6552 target points at 1024×256 ($\Delta s = 0.48\,\Delta x$), 1656 at 256×64 (the Python port's default): the top row first, then the bottom row, which is the whole trick of the update function.
- The generator prints `G` and `N_Top` because the update function needs them: "if you change the geometry or the resolution, you need to update the function … manually" (Lecture 3). The Python port reads `N_top` from the file instead (next slide).
- $\mu = 2.234$, $\rho = 1000$, $\Delta t = 2.5 \cdot 10^{-5}$, $T = 10$, a dump every 400 steps: an SI-flavoured set-up (water, a 12 m channel) that is dimensionless in effect.

THE SAWTOOTH PUMP: THE UPDATE FUNCTION

```
function targets =
     update_Target_Point_Positions(dt,
     current_time,targets)
freq = 2;          % pumping frequency
a = 0.337;         % total vertical distance
     the pump travels
N_top = 3276;      % # of points along top
yPTS = read_In_yPT_Positions('sawtooth.vertex
     ');
targets(1:N_top,3) = yPTS(1:N_top) ...
    - (a/2)*sin( 2*pi*freq*current_time );
```

The MATLAB function, abridged (`read_In_yPT_Positions` is its own helper).

```
# Python port: N_top from the file, the originals
     read once
def update_Target_Point_Positions(dt, current_time,
      targets):
    if _y0 is None:
        _original_positions()        # reads
     sawtooth.vertex; N_top = n // 2
    shift = (a / 2) * math.sin(
        2 * math.pi * freq * current_time)
    for i in range(_N_top):
        targets[i, 2] = _y0[i] - shift
    return targets
```

The Python port (`examples/IB2d/Sawtooth/update_Target_Point_Positions.py`).

THE UPDATE FUNCTION: WHAT IT DOES, AND THE TRAP IN IT

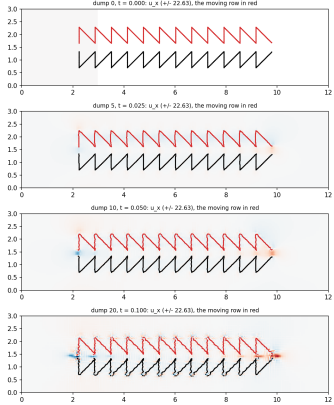
- The top row moves down and up as a sine of amplitude $a/2 = 0.1685$ at 2 cycles per unit time; the bottom row keeps its targets. The rows' tips are $G = 0.325$ apart, so the stroke ($a/2 \approx G/2$) brings the top row down to the channel's centreline and halves the gap (Lecture 3: "the gap is 0.325, and the pump will move down by a/2. Make sure the top and bottom don't collide"). The file's commented-out lines (`occ = 0.95`) belong to a version in which the bottom row rises too, for 95 % occlusion.
- The MATLAB version re-reads the `.vertex` file at every step ("`yPTS = read_In_yPT_Positions`"): 6552 lines, 400 000 times over the run. The port reads once.
- To move the bottom row too: "`targets(N_top+1:2*N_top,3) = yPTS(N_top+1:2*N_top) + movement_function`" (Lecture 3); in antiphase, the channel squeezes symmetrically.
- `N_top` is the hard-coded number that bites when the grid changes: 3276 at 1024×256, 828 at 256×64. A function that counts the file's points, or a generator that writes the count into a small file the function reads, removes the trap.

RUNNING THE SAWTOOTH ON A QUARTER OF THE GRID: THE STIFFNESS LESSON

- The MATLAB example is a 1024×256 grid for 400 000 steps: hours. The Python port takes the grid as an argument; at 256×64 (1656 points) a step takes 0.01–0.04 s on a desktop PC, so one pumping cycle ($T = 0.5$, 20 000 steps) would be a run of 4–14 minutes. None of the runs reached it.
- As shipped it blows up: with $k_t = 10^{11}$ and $\Delta t = 2.5 \cdot 10^{-5}$ the coarse run produces `NaN` positions at $t = 0.02$ (an `IndexError` from the delta-function indices is how Python reports it). The same Δt with $k_t = 10^9$ fails at $t = 0.03$, with 10^8 at $t = 0.065$ (and at $t = 0.068$ with $\Delta t = 10^{-5}$: not a time-step limit); 10^7 survives the longest, past $t = 0.12$, with the top row near the centreline; shorter strokes ($a = 0.25, 0.30$) with 10^7 fail at $t = 0.15$ and 0.13 (all times are those of the last dump before the failure).
- Softer is not the answer either: with $k_t = 10^7$ the worst point of the moving row is 0.09, two cells, from its target at $t = 0.1$ (the tooth tips are dragged sideways; the row's mean lag is a fifth of a cell), and the bottom row's tips have begun to shed points (next slide): the tether no longer holds the wall against the flow it drives.
- Nor is a shorter channel at the original resolution: four teeth in a 4×2 box on 384×192 ($\Delta x = 0.0104$, the original's 0.0117) with $k_t = 10^{11}$ ran past $t = 0.21$ and failed on the up-stroke; with 10^{10} at $t = 0.02$.
- The lesson: a pump whose walls come within a few cells of each other is the hardest case for a tethered boundary; this one was tuned for its grid, its stiffness and its step together, and no single knob transplants it to a laptop. Chapter 4d measures what each knob does on the rubber band; the working low-resolution pump of this chapter is the pulsing heart (slide 20).

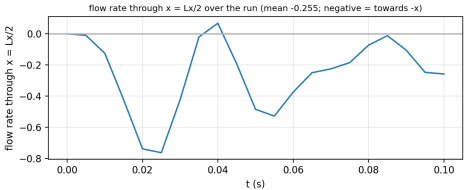
> Lecture 12's stability slides ("the time step necessary for stability is inversely proportional to the force applied to the fluid; the forcing term is typically what causes the instability, not the CFL condition") are chapter 4d, which checks the first claim on the rubber band.

THE SAWTOOTH PUMP: WHAT IT DOES



u_x at four times of the first tenth of a second (256×64, $k_t = 10^7$), the moving row in red; the colour scale is the largest $|u_x|$ of the four frames. The down-stroke pushes fluid out of both ends of the channel; by $t = 0.1$ the lower row's tips have begun to shed points (the tether is too soft to hold them), and the run fails at $t = 0.12$.

THE SAWTOOTH PUMP: THE FLOW IT DRIVES



- The flow rate through the channel's middle, $\int u_x \, dy$ at $x = L_x/2$, over the tenth of a second the run lasted: towards $-x$ after the first few dumps (one small positive excursion at $t = 0.04$), -0.76 at most, -0.26 on average. A full cycle, and so the net pumping, was not reached (previous slides). The MATLAB example's `Sawtooth_Data_Analysis.m` does the measurement differently: it averages the speed over a window at each end of the channel for the first three of the run's twenty cycles.
- The teeth make the two half-strokes unequal, which is how a symmetric sine of the wall pumps in one direction (Lecture 3's one-line description: "the top of the channel moves up and down to drive flow"). Chapter 4e has the pumps that do it with resting lengths and with muscles; the next slides have the one that runs on a laptop.

THE SAME MECHANISM FOR SPRINGS AND BEAMS

Resting lengths (`Tracers_In_Impedance_Pump`, Python; chapter 4b):

```
def update_Springs(dt, current_time, xLag, yLag, springs):
    # springs: columns 0-1 the two points, 2 k, 3 the resting
      length, 4 alpha
    ...                                        # choose the rows of one
       section of the tube
    springs[rows, 3] = L0 * (1 - amp * f(current_time))   # pinch:
       shorter rest length
    return springs
```

- The pinch is a shorter resting length for the springs across one section of the tube at the right phase; the walls pull in, the fluid is squeezed; tracers show the net transport (chapter 4b's picture).
- The heart tubes of `Example_HeartTube` do the same with a travelling wave ("dynamic suction pumping" and "peristaltic waves of contraction" in the example list) and, in the muscle versions, with an activation function instead (chapter 4e).

THE SAME MECHANISM FOR BEAMS

Rest curvature (the swimmer, MATLAB; chapter 4b):

```
function beams_info = update_nonInv_Beams(dt,current_time,
      beams_info)
% beams_info: col 1-3 the points, col 4 k_b, col 5 Cx, col 6 Cy
t = rem(current_time, period);      % time within the stroke
...                                  % blend the two stored
      phases at t
beams_info(:,5) = Cx_t;  beams_info(:,6) = Cy_t;
```

- The preferred shape is stored as two phases (`swimmer.phases`); the function blends them at the current time and writes the second differences into the curvature columns. The beam bends towards the new shape, the fluid pushes back, and the swimmer swims.
- `update_Beams` does the same with the scalar C of the torsional beams; `update_Damped_Springs` with k, L, b.
- Every table keeps its first columns (the indices) untouched: the connectivity is fixed for the run. A structure that changes topology (a valve that detaches) is not an `update_*` job.

INTERPOLATING BETWEEN PHASES (THE PULSING HEART)

- `Example_Pulsing_Heart`: a cartoon heart whose target points interpolate between two stored shapes ("interpolates between two phases", the example list), with a 32×32 and a 512×512 version; the swimmer stores two shapes the same way (their curvatures are computed in the function).
- The pattern: the generator writes the geometry at phase A as the `.vertex` file and the geometry at phase B as a second file (or a second column set); the update function reads both once, and sets the targets to $(1 - \theta(t))\,\mathbf{X}_A + \theta(t)\,\mathbf{X}_B$ with θ a smooth 0-to-1-to-0 function of the time within the period (the swimmer's quadratic–cubic–quadratic blend is one; $\frac{1}{2}(1 - \cos 2\pi t/P)$ is another).
- It is the second style of slide 9 with the geometry drawn twice instead of written as a formula: the way to prescribe a motion traced from images (a bell, a heart, a fin), with as many phases as the images give.
- The same two-state interpolation drives IBAMR's target points in chapter 9a; chapter 20's IBFE active stress prescribes a travelling wave instead of two shapes. What changes is only where the function lives.

EXAMPLE 3: THE PULSING HEART, IN PYTHON

```
tP1 = 0.025; tP2 = 0.025; period = tP1 + tP2     # contract, expand
def blend(s):                  # 0 -> 1 with zero slope at both ends
    if s < 0.1:  return a * s * s
    if s < 0.9:  return c * s**3 + d * s * s + gg * s + h
    return 1.0 - b * (s - 1.0)**2
def update_Target_Point_Positions(dt, current_time, targets):
    if _phases is None: _read_phases()          # All_Positions.txt,
       once
    x1, y1, x2, y2 = _phases
    t = current_time % period
    if t <= tP1:                                  # phase 1 -> phase
       2
        g = blend(t / tP1)
        for i in range(targets.shape[0]):
            targets[i, 1] = x1[i] + g * (x2[i] - x1[i])
            targets[i, 2] = y1[i] + g * (y2[i] - y1[i])
    else:                                         # phase 2 -> phase
       1
        g = blend((t - tP1) / tP2)
        for i in range(targets.shape[0]):
            targets[i, 1] = x2[i] + g * (x1[i] - x2[i])
            targets[i, 2] = y2[i] + g * (y1[i] - y2[i])
    return targets
```

- The update function of `examples/IB2d/Pulsing_Heart`, abridged: the phases are read once, the time within the period picks the half-beat, and every target is the blend of the two shapes.

THE PULSING HEART: THE FILES AND THE BLEND

- `Example_Pulsing_Heart/32x32`, ported (`examples/IB2d/Pulsing_Heart`): 384 target points ($k_t = 10^8$) tracing a heart with an opening at the top, in a 1×1 box on 32^2; $\mu = 0.5$, $\Delta t = 10^{-5}$, $T = 0.1$: two beats, a dump every 50 steps.
- `All_Positions.txt` holds the two shapes, one line per point (`x1 y1 x2 y2`); `heart.vertex` is phase 1. The function reads the file once (the MATLAB original re-reads it every step) and sets every target to the blend of the two phases at the time within the period: the second style, with the geometry drawn twice instead of written as a formula.
- The blend g is the quadratic–cubic–quadratic polynomial of the swimmer (same coefficients `a` to `h`); any smooth $0 \to 1$ function with zero end slopes does.

THE PULSING HEART: WHAT HAPPENS



- 10 000 steps in 24 s on a desktop PC; 201 dumps. The area inside the heart's outline (right) goes from 0.27 to 0.13 at the end of each contraction and back: the targets are followed exactly, beat after beat, because the motion is a function of time.
- The fluid is pushed out through the opening at the top during the contraction and drawn in during the expansion; at 32^2 the vorticity at the opening's tips reaches $7 \cdot 10^3$ (the colour scale), the kind of number that says "refine" (chapter 4d). The example's 512×512 folder is a different model (6829 points, springs as well, its own update function), not a refinement of this one.
- The whole example is the data file, the blend and forty lines: the recipe of slide 19 for any motion traced from images.

YOUR OWN EXAMPLE: FROM A SKETCH TO A FOLDER

1. Sketch the domain: the box $[0, L_x] \times [0, L_y]$ with the origin at the corner (chapter 4a), the structure inside it with room for the flow it makes, the periodic edges in mind (a wall you want needs target points).
2. Choose the grid: N_x, N_y even, $\Delta x = \Delta y$ (so $L_x/N_x = L_y/N_y$: the sawtooth's 1024×256 for 12×3); start coarse (32–64 per unit length) and refine later (chapter 4d).
3. Place the points at $\Delta s \approx \Delta x/2$ along the structure, by arc length, in an order you can describe ("points 0–827 are the top row"); print the counts the update function will need.
4. Decide the physics per point: springs between neighbours (stretch), beams over triples (bend), target points where it is held or driven, masses where inertia matters; write the files with full precision and the right index base (0 in Python, 1 in MATLAB).
5. Write `input2d` from a working example: the grid, μ, ρ, Δt and T, the flags that match the files you wrote, `string_name`, `print_dump`.
6. If it moves, write the `update_*` function in the second style (slide 9), with the counts read from the files.
7. Run short first: T of a few dumps, look at dump 0000 and 0001 (chapter 4b), then the console's "FIBER MODEL INCLUDES", then the full run.

THE COURSE'S TEMPLATE: TWO BEAMS DRIVEN BY TARGET POINTS

```
Lx = 1;  Ly = 2;  N = 512;              % the
     longest side has N cells
dx = Ly/N;  ds = Ly/(2*N);             % ds should
     be 1/2 of dx
Lb = Lx/2;  npts_beam = ceil(Lb/ds); % 256 points
      per beam
mesh_name = 'two_beams_';
for s = 0:npts_beam-1                  % the bottom
      beam, then the top one
   fprintf(vertex_fid, '%1.16e %1.16e\n', Lx/4+s*
     ds, Ly/2-Ly/8);
end
disp(['The vertex ids for the bottom beam go from
      1 to ' num2str(npts_beam)]);
```

`generate_meash.m` (sic), abridged.

```
amp = 1/8;  freq = 1;  stop_time = 6;
first_bottom = 1;  last_bottom = 64;  first_top
      = 257;  last_top = 320;
yPTS = read_In_yPT_Positions('two_beams_512.
     vertex');
while current_time < stop_time
    for i = first_bottom:last_bottom
      targets(i,3) = yPTS(i) - amp*sin( 2*pi*
     freq*current_time );
    end
    for i = first_top:last_top
      targets(i,3) = yPTS(i) + amp*sin( 2*pi*
     freq*current_time );
    end
end
```

`update_Target_Point_Positions.m`, abridged.

THE TEMPLATE: WHAT IT DOES, AND THREE THINGS TO FIX

`4-ib2d-MakeYourOwnExample` (the UA course, 2022): two horizontal beams of 256 points each in a 1×2 box, with springs, beams and target points (files `two_beams_512.*`), the beams' first quarters driven towards each other as a sine: a two-flap pump, with `read_Vertex_Points_and_Plot_Them.m` to check it. It shows the recipe of the previous slide, and three things to fix before it runs:

- `while` must be `if`: as written, the loop never ends when `current_time < stop_time` (nothing inside it changes the time): the run hangs in the first step.
- Rows are not indices: the generator writes target points for the first quarter of each beam only, so the `.target` file has 128 rows (rows 1–64 are points 1–64, rows 65–128 are points 257–320), but the function writes `targets(257:320, 3)`. MATLAB grows the table to 320 rows with zero indices, and the force routine then indexes `xLag(0)`: an error at the first step. Select rows 65–128, or the rows whose first column is between 257 and 320 (slide 6).
- The counts are typed in (`64`, `257`, `320`) for one resolution; the `disp` lines of the generator are there to set them.

Fixed, it is the simplest pump you can write: exercise 27.

The generator is called `generate_meash.m` in the folder; the files are in IB2d's MATLAB numbering.

CHECKS BEFORE AND AFTER THE FIRST RUN

- The geometry: the checker of chapters 4a and 6 (`test_IB2d_IBAMR_Geometry`, `test_ib_geometry.py`) with the right index base; a plot from your generator; $\Delta s/\Delta x$ printed.
- The files and flags agree: every file your generator wrote has its flag at 1 and the right `string_name`; every flag at 1 has its file (a missing file stops the run; a missing flag silently ignores the file: chapter 4a).
- The update function is found and used: the console's "dynamically updating target point properties" line; dump 0001 differs from dump 0000 where it should.
- The counts: print them from the function once (`N_top`, the rows selected) and compare with the generator's output.
- The motion: plot the targets' path from the function alone, without the fluid (the swimmer's `test_Script.m` does this; a five-line script that calls your function for a few times does the same), before spending the run.
- Stability: the first dumps at the chosen Δt; if the band or wall explodes, halve Δt or soften the stiffness (slide 14; chapter 4d).
- Resolution: the same run at twice the grid, once, before any number from it is reported (chapter 4d).

EXERCISES

1. The course's homework: modify the rubber band so that it is 50 % smaller and deformed differently, with your own generator (Lectures 3 and 9); then drive it along a circle with an update function written in the second style (position as a function of time), and compare the band's centre with the prescribed path.
2. On the full grid (MATLAB, or Python on a cluster), move the sawtooth's bottom row in antiphase (Lecture 3's line) and compare the flow rate through the middle with the one-row pump; then set the top row's amplitude so that its tips reach the bottom row's ($a = 2G$) and see what the occlusion does.
3. Port the two-beam template to Python with the three fixes of slide 25: a 256-point beam each, the first quarter of each driven; run it at 128×256 and plot the flow rate between the beams.

EXERCISES (2)

4. Take the moving band and halve Δt: compare the band's centre at $t = 0.025$ between the two runs for the increment style, then rewrite the function in the second style and compare again.
5. Draw two phases of a shape of your own (a bell, a fin, a chamber), write them as the pulsing heart's four-column file, and run it with the same function; then change the blend to $\frac{1}{2}(1 - \cos 2\pi t/P)$ and compare the vorticity at the opening.
6. Design an example of your own from the project ideas of Lecture 2 (a barb moving towards a circle, a heart tube that pumps, a leaf that vibrates in a flow) following slide 23; hand in the sketch, the generator, the function and the first two dumps.

Lecture 11's homework 4, option 1: run an example related to your project instead. Appendix C collects the homework bank.

SUMMARY

- A boundary moves in IB2d because something it is tied to changes in time: a target position, a resting length, a rest curvature, an activation. Each has a flag in `input2d` and a function in the example folder that the driver calls every step with the time and the current table, and whose returned table replaces it.
- The moving band increments the targets every step; the sawtooth sets them from the original positions as a function of time. Use the second style: no drift, the same motion at every Δt.
- The sawtooth's hard-coded `N_top` and the two-beam template's `while` and row numbers are the classic mistakes of this kind of file: counts typed in for one resolution, a loop that was meant to be a condition, rows of the table confused with indices of the points. Read the counts from the files; test the function without the fluid.
- Coarsening a grid changes what a tethered wall can take: the sawtooth pump, built for 1024×256, failed on every coarser grid tried on a desktop PC (stiff tethers blow up, soft ones let points stray two cells from their targets), while the pulsing heart at 32^2 runs in half a minute; chapter 4d makes that a measurement.
- A new example is a sketch, a grid, points at $\Delta x/2$ in a known order, files with the right index base, an `input2d` copied from a working example, a short first run and a look at the first dumps.
- Next: chapter 4d, the stability and convergence lab.

REFERENCES AND CREDITS

- The IB2d repository, https://github.com/nickabattista/IB2d: `matIB2d/Examples/Example_Moving_Rubberband` (`Rubberband.m`, `update_Target_Point_Positions.m`), `Example_Sawtooth` (`give_Me_Sawtooth_Geometry.m`, `update_Target_Point_Positions.m`, `input2d`, `Sawtooth_Data_Analysis.m`), `Example_Pulsing_Heart/32x32` (`All_Positions.txt`, `update_Target_Point_Positions.m`), `Examples_First_Year_Seminar/Swimmer`; `pyIB2d/Examples/Tracers_In_Impedance_Pump/update_Springs.py`; the drivers' calls to the update functions (`IBM_Driver.m` lines 805–821, `IBM_Driver.py` lines 697–721). Code as of the 2024-03-06 clone.
- The Python ports and runs (October 2026): `examples/IB2d/Moving_Rubberband`, `Sawtooth` and `Pulsing_Heart` in this repository (the sawtooth variants tried: `wsl/final/ch4c_saw*.sh`); `wsl/final/ch4c_*.sh`; figures by `tools/ch4c_figures.py` with `examples/analysis/read_vtk_legacy.py`.
- Sources merged (slide 3): "Changing target positions" (Lecture 3, UA 2022), Lectures 11 and 12 (2021–23), the 2024 practical (slide 29), "IB2d and IBAMR associated files" (Lecture 2, 2022), the UA "4-ib2d-MakeYourOwnExample" folder, "Your own rubber band" (the IBAMR Lecture 3 and Lecture 9 homework). Quotations are from those decks.